

Module 2 – Introduction to Programming

Overview of C Programming

1. THEORY EXERCISE: Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.

➤ The History and Evolution of C Programming

- The C programming language is one of the most important and influential programming languages in computer history. It was created in 1972 by Dennis Ritchie at Bell Labs. C was developed to improve an earlier language called B, and it was mainly used to build the Unix operating system.

➤ Importance of C Programming

- C is important because it is powerful, fast, and flexible. It gives programmers direct control over computer memory and hardware. This makes it very useful for:
- Operating systems (like Unix)
- Embedded systems (such as microcontrollers and devices)
- Compilers
- Databases
- Many modern operating systems, including Linux, are written mainly in C.

➤ Why C Is Still Used Today

- Even though C is more than 50 years old, it is still widely used today. There are several reasons for this:
- Speed and Performance – C programs run very fast.
- Portability – C programs can run on many different types of computers with little change.
- Low-Level Access – C allows direct access to memory and hardware.
- Foundation for Other Languages – Many modern languages are based on C.

LAB EXERCISE: Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development.

2. THEORY EXERCISE: Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

➤ Step 1: Install a C Compiler (GCC)

- A compiler changes your C code into a program the computer can run. One popular compiler is GNU Compiler Collection

☒ For Windows:

Option 1: Install using Dev-C++

Download and install Dev-C++.

During installation, make sure GCC compiler is selected.

After installation, GCC is ready to use.

Option 2: Install using Code::Blocks

Download the version that says "with MinGW".

Install it.

This version already includes GCC.

☒ For Mac:

Open Terminal.

Type:

```
xcode-select --install
```

☒ For Linux:

Open Terminal.

Type:

```
sudo apt install build-essential
```

Press Enter.

GCC will be installed.

◆ Step 2: Set Up an IDE (Easy Code Editor)

An IDE helps you write, run, and manage your code easily.

◆ Option 1: Using Dev-C++ (Easy for Beginners)

Open Dev-C++.

Click File → New → Source File.

Write your C program.

Click Compile & Run.

◆ Option 2: Using Code::Blocks

Open CodeBlocks.

Click Create New Project → Console Application → C.

Write your code.

Click Build and Run.

LAB EXERCISE: Install a C compiler on your system and configure the IDE. Write your first program to print "Hello, World!" and run it.

3. THEORY EXERCISE: Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples

➤ Basic Structure of a C Program

- A C program has some important parts:
- Headers
- Main Function
- Comments
- Data Types
- Variables

➤ Headers

- Headers are files that include built-in functions.
- They are written at the top of the program.

➤ Example:

- `#include <stdio.h>`
- `#include` is used to include a header file.
- `stdio.h` is used for input and output functions like `printf()` and `scanf()`.

➤ Main Function

- Every C program must have a main function.
- The program starts running from `main()`.

➤ Example:

```
int main() {  
    return 0;  
}
```

- `int` means the function returns a number.
- `return 0;` means the program ended successfully.

➤ Comments

- Comments are notes in the program.
- They are ignored by the compiler.

➤ Single-line comment:

```
// This is a comment
```

➤ Multi-line comment:

```
/* This is
```

a multi-line comment */
Comments help explain the code.

- Data Types
 - Data types tell the computer what kind of data we are storing.
- Common data types:
 - int → whole numbers (10, 25)
 - float → decimal numbers (3.5, 7.2)
 - char → single character ('A', 'b')
 - double → large decimal numbers
- Example:
 - int age = 20;
 - float price = 99.5;
 - char grade = 'A';
- Variables
 - Variables are used to store data.
 - We must declare a variable before using it.
- Example:
 - int number;
 - number = 5;
 - Or we can declare and assign together:
 - int number = 5;

LAB EXERCISE: Write a C program that includes variables, constants, and comments. Declare and use different data types (int, char, float) and display their values.

4. THEORY EXERCISE: Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.

➤ Arithmetic Operators

These are used to perform mathematical calculations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b

- Example:
- ```
int a = 10, b = 5;

printf("%d", a + b); // Output: 15
```

➤ Relational Operators

- These are used to compare two values.
- The result is true (1) or false (0).

| Operator | Meaning                  |
|----------|--------------------------|
| ==       | Equal to                 |
| !=       | Not equal to             |
| >        | Greater than             |
| <        | Less than                |
| >=       | Greater than or equal to |
| <=       | Less than or equal to    |

➤ Example:

```
int a = 10, b = 5;

printf("%d", a > b); // Output: 1 (true)
```

➤ Logical Operators

- These are used to combine conditions.

| Operator | Meaning |
|----------|---------|
| &&       | AND     |
| !        | NOT     |

➤ Example:

```
int a = 10, b = 5;

printf("%d", (a > 5 && b < 10)); // Output: 1
```

➤ Assignment Operators

- These are used to assign values to variables.

| Operator | Example | Meaning   |
|----------|---------|-----------|
| =        | a = 5   | Assign    |
| +=       | a += 2  | a = a + 2 |
| -=       | a -= 2  | a = a - 2 |
| *=       | a *= 2  | a = a * 2 |
| /=       | a /= 2  | a = a / 2 |

➤ Example:

```
int a = 5;

a += 3; // Now a = 8
```

- Increment and Decrement Operators
  - These increase or decrease a value by 1.

| Operator | Meaning       |
|----------|---------------|
| ++       | Increase by 1 |
| --       | Decrease by 1 |

- Example:
 

```
int a = 5;
a++; // a becomes 6
a--; // a becomes 5
```
- Bitwise Operators
  - These work on binary (bits).

| Operator | Meaning     |
|----------|-------------|
| &        | AND         |
| `        | `           |
| ^        | XOR         |
| ~        | NOT         |
| <<       | Left shift  |
| >>       | Right shift |

- Example:
 

```
int a = 5, b = 3;

printf("%d", a & b); // Bitwise AND
```

LAB EXERCISE: Write a C program that accepts two integers from the user and performs arithmetic, relational, and logical operations on them. Display the results.

5. Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.
  - Decision-making statements help the program choose what to do based on a condition.
    - The main decision statements are:
      - if
      - if-else
      - Nested if-else
      - Switch
  - 1. if Statement
    - The if statement checks a condition.
    - If the condition is true, the code inside it runs.
      - Syntax:
 

```
if (condition) {
 // code
```

```

}
▪ Example:
#include <stdio.h>
int main() {
 int age = 18;
 if (age >= 18) {
 printf("You are an adult");
 }
 return 0;
}

```

If the condition is true, it prints the message.

## ➤ 2. if-else Statement

- The if-else statement has two choices.
- If the condition is true → run if block.
- If false → run else block.

```

▪ Syntax:
 if (condition) {
 // true block
 } else {
 // false block
 }

▪ Example:
#include <stdio.h>
int main() {
 int number = 5;
 if (number % 2 == 0) {
 printf("Even number");
 } else {
 printf("Odd number");
 }
 return 0;
}

```

## ➤ 3 . Nested if-else

- Nested means if inside another if.
- It is used when we check multiple conditions.

```

▪ Example:
#include <stdio.h>
int main() {
 int marks = 85;
 if (marks >= 50) {
 if (marks >= 75) {
 printf("Distinction");
 } else {
 printf("Pass");
 }
 } else {

```

```
printf("Fail");
}
return 0;
```

➤ 4. switch Statement

- The switch statement is used when we have many choices for one variable.
- It is easier than writing many if-else statements.

- Syntax:

```
switch (variable) {
 case value1:
 // code
 break;
 case value2:
 // code
 break;
 default:
 // code
}
```

- Example:

```
#include <stdio.h>
int main() {
 int day = 2;
 switch (day) {
```

```
case 1:
```

```
 printf("Monday");
 break;
```

```
case 2:
```

```
 printf("Tuesday");
 break;
```

```
case 3:
```

```
 printf("Wednesday");
 break;
```

```
default:
```

```
 printf("Invalid day");
```

```
}
```

```
return 0;
```

```
}
```

6. Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

➤ Loops in C

- Loops are used to repeat a block of code multiple times.
- The three main loops are:
  - while
  - for
  - do-while



➤ 1. while Loop

- The while loop checks the condition before running the code.

- Syntax:

```
while (condition) {
 // code
}
```

- Example:

```
int i = 1;
while (i <= 5) {
 printf("%d\n", i);
 i++;
}
```

➤ 2. for Loop

- The for loop is mostly used when you know how many times the loop should run.

- Syntax:

```
for (initialization; condition; update) {
 // code
}
```

- Example:

```
for (int i = 1; i <= 5; i++) {
 printf("%d\n", i);
}
```

➤ 3. do-while Loop

- The do-while loop runs the code at least once, then checks the condition.

- Syntax:

```
do {
 // code
} while (condition);
```

- Example:

```
int i = 1;
do {
 printf("%d\n", i);
 i++;
} while (i <= 5);
```

LAB EXERCISE: Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while).

7. THEORY EXERCISE: Explain the use of break, continue, and goto statements in C. Provide examples of each.

➤ Control Statements in C

- These statements are used to control the flow of loops and programs.
- break
- continue

- goto

### ➤ 1. break Statement

- The break statement is used to stop a loop immediately.
- It is mostly used in loops and switch statements.

#### ▪ Example (Loop):

```
#include <stdio.h>
int main() {
 for (int i = 1; i <= 5; i++) {
 if (i == 3) {
 break; // stops the loop
 }
 printf("%d\n", i);
 }
 return 0;
}
```

#### ▪ Output:

1

2

When i becomes 3, the loop stops.

### ➤ 2. continue Statement

- The continue statement skips the current loop iteration and moves to the next one.

#### ▪ Example:

```
#include <stdio.h>
int main() {
 for (int i = 1; i <= 5; i++) {
 if (i == 3) {
 continue; // skip number 3
 }
 printf("%d\n", i);
 }
 return 0;
}
```

#### Output:

1

2

4

5

When i is 3, it skips printing 3.

### ➤ 3. goto Statement

- The goto statement jumps to a labeled part of the program.

- Syntax:  
`goto label;`  
`...`  
`Label:`  
`statement;`
  
- Example:  

```
#include <stdio.h>
int main() {
 int i = 1;
start:
 if (i <= 5) {
 printf("%d\n", i);
 i++;
 goto start; // jump back
 }
 return 0;
}
```

LAB EXERCISE: Write a C program that uses the break statement to stop printing numbers when it reaches 5. Modify the program to skip printing the number 3 using the continue statement.

8. THEORY EXERCISE: What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

- What Are Functions in C?
  - A function is a block of code that performs a specific task.
  - Functions help:
    - Divide the program into small parts
    - Reuse code
    - Make programs easier to understand
- Parts of a Function
  - There are three main parts of a function:
    - Function Declaration
    - Function Definition
    - Function Call
- 1. Function Declaration
 

A function declaration tells the compiler:

  - Function name
  - Return type
  - Parameters (inputs)
  - It is written before main().

- Syntax:

- `return_type function_name(parameters);`

- Example:

- `int add(int a, int b);`
- This means:
- Function name: add
- Takes two integers
- Returns an integer

➤ 2. Function Definition

- The function definition contains the actual code.

- Syntax:

```
return_type function_name(parameters) {
 // code
}
```

- Example:

```
int add(int a, int b) {
 return a + b;
}
```

➤ 3. Function Call

- A function call is used to run the function.
- It is usually written inside `main()`.

- Example:

```
int result = add(5, 3);
```

This calls the function and stores the result.

➤ Complete Example

```
#include <stdio.h>
```

```
// Function declaration
```

```
int add(int a, int b);
```

```
int main() {
```

```
 int result;
```

```
 // Function call
```

```
 result = add(5, 3);
```

```

 printf("Sum = %d", result);

 return 0;
}

```

// Function definition

```

int add(int a, int b) {
 return a + b;
}

```

Output:

Sum = 8

LAB EXERCISE: Write a C program that calculates the factorial of a number using a function. Include function declaration, definition, and call.

9. THEORY EXERCISE: Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

➤ What Is an Array in C?

- An array is a collection of elements of the same data type stored in continuous memory locations.
- It is used to store multiple values using one variable name.

- Example:

Instead of writing:

```

int a1 = 10;
int a2 = 20;
int a3 = 30;

```

- We can write:

```

int numbers[3] = {10, 20, 30};

```

➤ 1.One-Dimensional Array (1D Array)

- A one-dimensional array is like a list.

- Syntax:

```
data_type array_name[size];
```

- Example:

```
#include <stdio.h>

int main() {

 int numbers[3] = {10, 20, 30};

 printf("%d\n", numbers[0]); // 10
 printf("%d\n", numbers[1]); // 20
 printf("%d\n", numbers[2]); // 30
 return 0;

}
```

## ➤ 2. Multi-Dimensional Array

- A multi-dimensional array has more than one dimension.
- The most common type is a 2D array.
- It is like a table (rows and columns).
- Two-Dimensional Array (2D Array)

- Syntax:

```
data_type array_name[rows][columns];
```

- Example:

```
#include <stdio.h>

int main() {
 Int matrix[2][2] = {

 {1, 2},

 {3, 4}

 };

 printf("%d\n", matrix[0][0]); // 1
```

```
printf("%d\n", matrix[1][1]); // 4

return 0;
}
```

LAB EXERCISE: Write a C program that stores 5 integers in a one-dimensional array and prints them. Extend this to handle a two-dimensional array (3x3 matrix) and calculate the sum of all elements.

10. THEORY EXERCISE: Explain what pointers are in C and how they are declared and initialized.  
Why are pointers important in C?

